

Layer 4

Reasoning Model

• Spending additional time during generation

→ they don't predict next words. they Generate multiple hidden paths → test them against internal Reward

* a common misconception they are not Just output Really
Long explanation

test-to-compute

• Standard mode Spent all their Computational Budget during the initial training

→ Reasoning model introduces test-to-time compute ? Spending extra Resource at the exact moment you ask Question

This extra Resource spends on:

- Generating multiple Potential candidate answer
- Voting across the answers
- Run Search over Reasoning Paths (Like chess engine)
- Execute code or using tools to test hypotheses
- critiquing it's own draft and Revising Before outputting

CoT : Chain-of-Thought

Forcing the model to break a problem down into Step-by-step Reasoning before giving answer

- math
- Logic puzzles
- coding

and multi-step planning

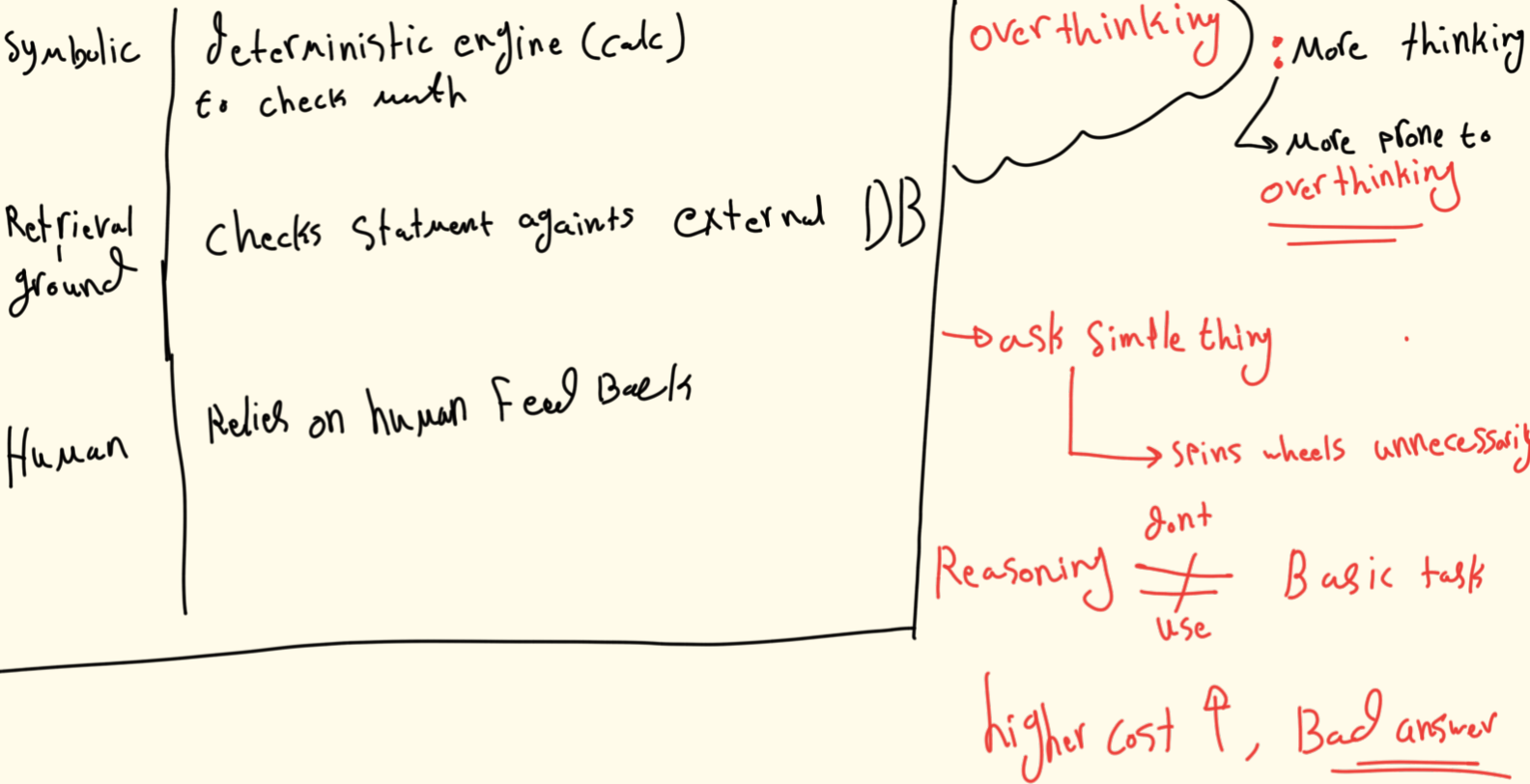
CoT Failure :

- Fabrication : hallucinate a logical-sounding step that factually wrong
- Knowledge gaps : Require Real-world Data doesn't have, but tries Logic it any way
- Persuasive but wrong : the chain of reasoning sounds incredibly convincing, masking a fundamentally flawed conclusion
- Hidden assumption : the model makes an unchecked leap in logic early on, poisoning the rest of the chain

Verifiers

- Reasoning model has multiple paths
- Which one is the Best? : Verifiers

outcome	check final answer	- a system that check candidate <u>answers</u> ,
Process	evaluate the logic of each individual step	- Verifiers works best where domain correctness is objective and <u>measurable</u> (math, code)
unit test	Runs code against test cases	



~~Only~~ use Reasoning model

When ^{extra} test-time compute actually
changes the accuracy of the outcome

~~Don't~~ use it every time as default
Prompt